INSTITUT NATIONAL POLYTECHNIQUE FÉLIX HOUPHOUËT-BOIGNY
Yamoussoukro

Master 2 Data Sciences — Année universitaire 2026-2027

Cours de Java SE — Aspects avancés
Prof. Bi Tra GOORE



RAPPORT DE PROJET

Simulation du traitement d'une élection législative

Application JavaFX « Élection législative de Nepalus »

Réalisé par :

DAH BEBE SAMUEL
BROU KOUAKOU CEPHAS
DJE KOFFI EUDES

Enseignant : Prof. Bi Tra GOORE

Date de remise : 13 novembre 2026

Sommaire

1. Introduction 3
2. Analyse du problème 4
2.1 Règles de l'élection 4
2.2 Fonctionnalités attendues 4
2.3 Données générées aléatoirement 4
2.4 Questions auxquelles l'interface doit répondre 4
2.5 Conséquences des règles, à noter 5
3. Conception 6
3.1 Architecture générale 6
3.2 Interfaces 6
3.3 Classes abstraites 6
3.4 Classes concrètes 7
3.5 Diagrammes de classes 8
4. Réalisation 10
4.1 Environnement de développement 10
4.2 Modèle métier et respect des règles 10
4.3 Génération aléatoire des résultats 10
4.4 Persistance des données 11
4.5 Interface graphique 11
4.6 Traitements en arrière-plan 14
5. Notions du cours mises en œuvre 15
6. Tests et validation 16
7. Guide d'utilisation 17
7.1 Lancement 17
7.2 Scénario d'utilisation 17
8. Difficultés rencontrées, limites et perspectives 18
8.1 Difficultés rencontrées 18
8.2 Limites 18
8.3 Perspectives 18
9. Conclusion 19

1. Introduction

Ce rapport présente le projet de Java SE réalisé dans le cadre du Master 2 Data Sciences de l'INP-HB. Le sujet consiste à **simuler le traitement informatique d'une élection législative** dans un pays fictif, Nepalus, découpé en départements. Chaque parti politique présente un candidat dans chaque département, et un député est élu par département.

Le travail demandé comporte trois parties :

1. identifier, décrire et définir les interfaces, les classes abstraites et les classes concrètes de l'application, et les présenter sous forme de diagramme de classes ;
2. développer l'application ;
3. rendre un rapport de projet ainsi que l'archive exécutable (fichier `.jar`).

Nous avons réalisé une application de bureau en **Java 17** avec une interface graphique **JavaFX 21** organisée selon le modèle MVC (vues FXML et contrôleurs). Les données sont persistantes : elles sont enregistrées dans des **fichiers CSV**, comme le demande l'énoncé. En complément, elles peuvent aussi être enregistrées dans une **base de données MySQL**, ce qui met en œuvre les notions JDBC vues en travaux dirigés.

Le rapport suit la démarche du projet : analyse du problème (section 2), conception et diagrammes de classes (section 3), réalisation (section 4), notions du cours mises en œuvre (section 5), tests (section 6), guide d'utilisation (section 7), puis difficultés rencontrées et perspectives (section 8).

2. Analyse du problème

2.1 Règles de l'élection

En plus des règles habituelles d'une élection, l'énoncé impose les règles suivantes :

- **Pas de bulletin blanc** : chaque votant choisit obligatoirement un candidat. Le total des voix d'un département est donc égal à son nombre de votants.
- **Pas d'égalité** : dans un même département, deux candidats ne peuvent pas obtenir le même nombre de voix.
- **Scrutin majoritaire à un tour** : le candidat qui obtient le plus grand nombre de voix est élu.

Nous en avons déduit une règle de cohérence évidente : le nombre de votants d'un département ne peut pas dépasser son nombre d'inscrits.

2.2 Fonctionnalités attendues

Fonctionnalité de l'énoncé	Réponse apportée dans l'application
Enrôlement des départements et des partis politiques	Onglets *Départements* et *Partis politiques* : formulaire d'enrôlement, retrait, contrôle des doublons (numéro, sigle).
Affichage de la liste des départements et des partis	Tableaux (`TableView`) triables, avec champ de recherche insensible aux accents.
Enregistrement des résultats du vote	Saisie manuelle par département, ou génération aléatoire pour un département ou pour tout le pays ; enregistrement dans le stockage actif (Ctrl+S).
Affichage des résultats par département	Onglet *Résultats par département* : classement des candidats, voix, pourcentages, élu et diagramme circulaire.
Questions d'analyse (voir 2.4)	Onglet *Analyses*, organisé en sections repliables.

2.3 Données générées aléatoirement

Pour chaque département, l'application tire au hasard :

- le **nombre d'inscrits**, entre un minimum et une **borne supérieure** que l'utilisateur choisit ;
- le **nombre de votants**, qui ne dépasse pas le nombre d'inscrits ;
- la **répartition des votants entre les candidats**, dont la somme est exactement égale au nombre de votants et dont les valeurs sont toutes différentes.

2.4 Questions auxquelles l'interface doit répondre

Question	Méthode du modèle	Affichage
Départements où au moins 50 % des inscrits ont voté	`Election.departementsAvecParticipationAuMoins(0.5)`	Analyses, avec un curseur pour faire varier le seuil de 0 à 100 %
Parti gagnant dans chaque département	`Election.partiGagnant(d)` → `RegleScrutin.designerElu(d)`	Départements (colonne *Élu*),

		Résultats, Analyses
Parti ayant le plus mauvais score dans chaque département	`Election.partiDernier(d)` → `RegleScrutin.designerDernier(d)`	Analyses (colonne *Plus mauvais score*)
Départements où tous les inscrits ont voté, et pour le même parti	`Election.departementsUnanimes()`	Analyses (section dédiée, voir 2.5)
Taux de participation par département et taux général	`Departement.getTauxParticipation()`, `Election.tauxParticipationGeneral()`	Départements, bandeau supérieur, Analyses
« Etc. » : questions supplémentaires	`siegesParParti()`, `partiMajoritaire()`, `voixParParti()`, `statistiquesParticipation()`	Analyses : composition de l'Assemblée, majorité absolue, moyenne, médiane, écart-type et histogramme des taux de participation

Le taux général est calculé comme le rapport entre le total des votants et le total des inscrits du pays, et non comme la moyenne des taux départementaux, qui donnerait le même poids à un petit et à un grand département.

2.5 Conséquences des règles, à noter

L'analyse des règles a fait apparaître deux conséquences qui ont guidé la conception.

a) Un département « unanime » est impossible dès trois partis. Si tous les inscrits votent pour le même parti, tous les autres candidats ont 0 voix. Avec trois partis ou plus, au moins deux candidats sont alors à égalité (0 voix chacun), ce que la règle « pas d'égalité » interdit. La question « départements où tous les électeurs ont voté pour le même parti » n'a donc de réponse non vide qu'avec **exactement deux partis**. L'application calcule bien cette liste (et la teste), et l'interface indique « Aucun département dans ce cas » lorsqu'elle est vide.

b) Il faut un minimum de votants. Pour que n candidats aient des scores tous différents, le plus petit total possible est $0 + 1 + \dots + (n - 1) = n(n - 1)/2$ voix. Avec 5 partis, il faut donc au moins 10 votants dans chaque département. Le générateur refuse une borne d'inscrits inférieure à ce minimum et l'explique à l'utilisateur.

3. Conception

3.1 Architecture générale

Le code est découpé en quatre paquetages, du plus indépendant au plus dépendant :

Paquetage	Rôle
`ci.inphb.election.modele`	Modèle métier : entités de l'élection, règle du scrutin, questions d'analyse. Ne dépend ni de JavaFX ni du stockage.
`ci.inphb.election.simulation`	Génération aléatoire des résultats et données de démonstration.
`ci.inphb.election.persistance`	Enregistrement et chargement de l'élection : fichiers CSV ou base MySQL.
`ci.inphb.election.vue`	Interface JavaFX : contrôleurs des vues FXML, contexte partagé, dialogues.

Ce découpage respecte le modèle **MVC** : le modèle ignore l'interface ; les vues sont décrites en FXML (`principale.fxml`, qui inclut `departements.fxml`, `partis.fxml`, `resultats.fxml` et `analyses.fxml` par `fx:include`) ; les contrôleurs font le lien entre les deux.

3.2 Interfaces

Interface	Description
`Identifiable`	Tout objet persistant possède un code unique (`getCode()`) : numéro de département sur trois chiffres, sigle de parti, ou couple département/parti pour un candidat.
`RegleScrutin`	Règle électorale appliquée dans chaque département : `valider(d)`, `designerElu(d)`, `designerDernier(d)`. Une autre règle (scrutin à deux tours, par exemple) pourrait être ajoutée sans modifier le reste de l'application.
`Depot<T extends Identifiable>`	Interface **générique** d'accès aux données : `chargerTout()` et `sauvegarderTout(Collection<? extends T>)`.
`StockageElection`	Lieu où l'élection complète est enregistrée : `charger()`, `sauvegarder(e)`, `contientDesDonnees()`, `getDescription()`.
`Navigation`	Déplacements entre onglets demandés par l'utilisateur (clic sur un graphique, double-clic sur une ligne). Implémentée par une classe anonyme du contrôleur principal.

3.3 Classes abstraites

Classe abstraite	Description
`Entite`	Implémente `Identifiable` et `Comparable<Entite>`. Factorise le nom, l'égalité (fondée sur le code), le tri et le contrôle des champs texte. Méthode abstraite : `getLibelle()`.
`StockageAbstrait`	Implémente `StockageElection`. Contient la logique commune de lecture et d'écriture d'une élection à partir de trois dépôts (partis, départements, résultats), quel que soit le support.
`DepotFichierCsv<T>`	Dépôt générique sur un fichier CSV. Lit et écrit le fichier ; les sous-classes ne fournissent que l'en-tête et la conversion objet ⇄ champs (`versChamps`, `depuisChamps`).

`DepotJdbc<T>`	Dépôt générique sur une table SQL. Exécute les requêtes ; les sous-classes fournissent le texte des requêtes et la conversion objet ⇄ ligne (`depuisLigne`, `renseigner`).
`ControleurOnglet`	Base commune des contrôleurs d'onglets : accès au contexte, méthode abstraite `rafraichir()`, méthodes génériques de création de colonnes, de filtrage et de cellules colorées.

`DepotFichierCsv` et `DepotJdbc` appliquent le principe du **patron de méthode** : l'algorithme général est écrit une seule fois dans la classe abstraite, et seules les étapes propres à chaque type d'objet sont laissées aux sous-classes.

3.4 Classes concrètes

Classe	Description
`Departement`	Hérite d'`Entite`. Numéro, nom, inscrits, votants, liste de ses candidats ; taux de participation ; état « dépouillé ».
`PartiPolitique`	Hérite d'`Entite`. Sigle (identifiant) et nom.
`Candidat`	Implémente `Identifiable`. Lien entre un département et un parti, avec le nombre de voix obtenues et le pourcentage des suffrages.
`Election`	Point d'entrée du modèle : enrôlement, enregistrement des résultats (validé par la règle de scrutin) et toutes les questions d'analyse.
`ScrutinMajoritaireUnTour`	Implémente `RegleScrutin` : vérifie l'absence de vote blanc, l'absence d'égalité et votants ≤ inscrits ; désigne l'élu et le dernier.
`StatistiquesParticipation`	Moyenne, médiane, écart-type, minimum, maximum et histogramme des taux de participation.
`ElectionException`, `PersistanceException`	Exceptions contrôlées propres au métier et au stockage.
`GenerateurResultats` (+ classe interne `Tirage`)	Tirage aléatoire des inscrits, votants et voix.
`DonneesDemo`	Jeu de 12 départements et 5 partis fictifs.
`StockageFichiers`, `StockageBaseDeDonnees`	Les deux supports de stockage, sous-classes de `StockageAbstrait`.
`DepotPartis`, `DepotDepartements`, `DepotResultats`	Dépôts CSV concrets (T = `PartiPolitique`, `Departement`, `Candidat`).
`DepotJdbcPartis`, `DepotJdbcDepartements`, `DepotJdbcResultats`	Dépôts JDBC concrets, pour les mêmes trois types.
`ConfigurationStockage` (+ énumération `Mode`), `ParametresConnexion`	Choix du stockage et paramètres MySQL, mémorisés dans `election.properties`.
`ApplicationElection`, `Lanceur`	Démarrage de l'application JavaFX (écran d'accueil, fenêtre principale). `Lanceur` permet d'exécuter le `.jar` sans configuration de modules.
`ControleurPrincipal`, `ControleurDepartements`, `ControleurPartis`, `ControleurResultats`, `ControleurAnalyses`	Contrôleurs des vues FXML (menus et un contrôleur par onglet).

`ContexteApplication`	État partagé entre les contrôleurs, exposé par des **propriétés observables** (révision du modèle, données modifiées, stockage actif).
`DialogueConnexion`, `IndicateurAnime`, `Animations`, `Palette`	Composants d'interface : dialogue des paramètres MySQL, indicateurs chiffrés animés, effets visuels, couleurs des partis.

3.5 Diagrammes de classes

Les diagrammes ont été réalisés avec PlantUML (source : `docs/diagramme-classes.puml`). Le premier présente le modèle métier, le second l'architecture d'ensemble (persistance, simulation, interface).

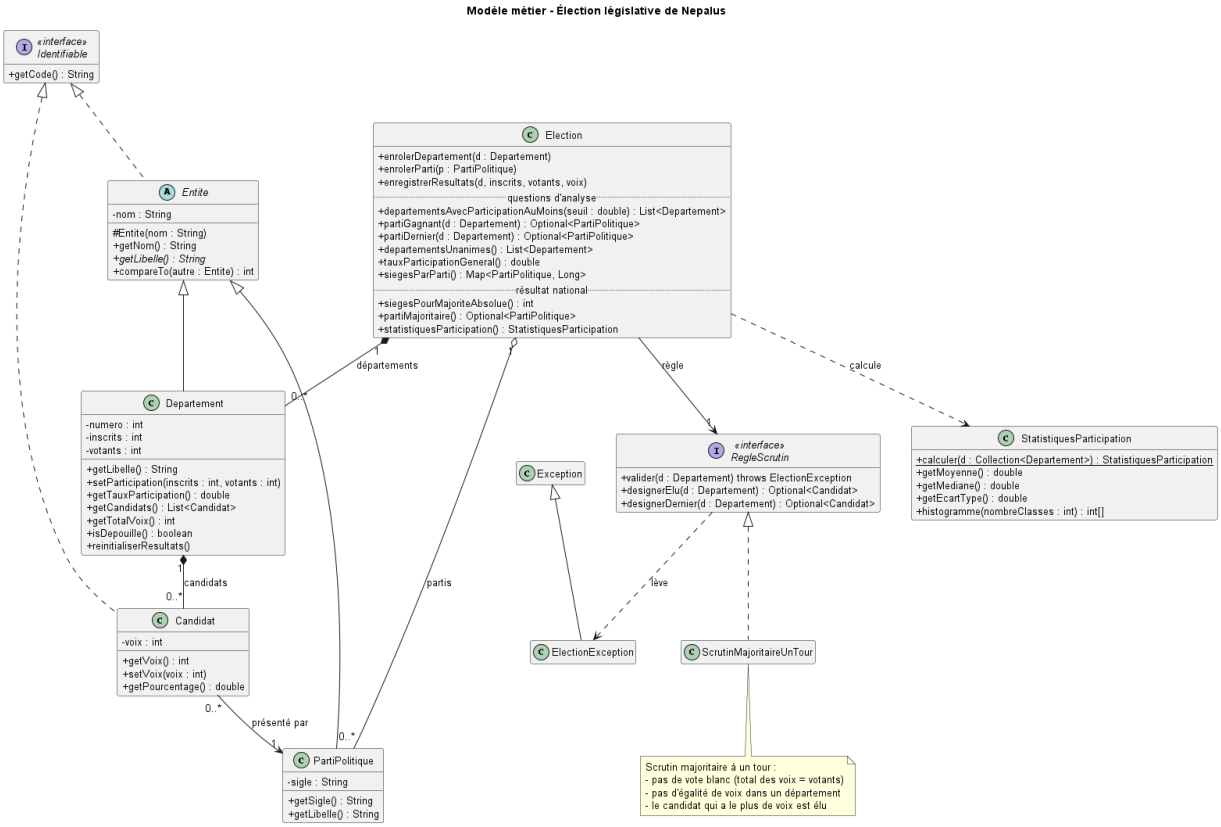


Figure 1 — Diagramme de classes du modèle métier

On y lit les relations principales : une `Election` est composée de départements et agrège des partis ; chaque `Departement` est composé de ses candidats ; chaque `Candidat` est présenté par un `PartiPolitique` ; l'élection délègue la validation et la désignation de l'élu à sa `RegleScrutin`.

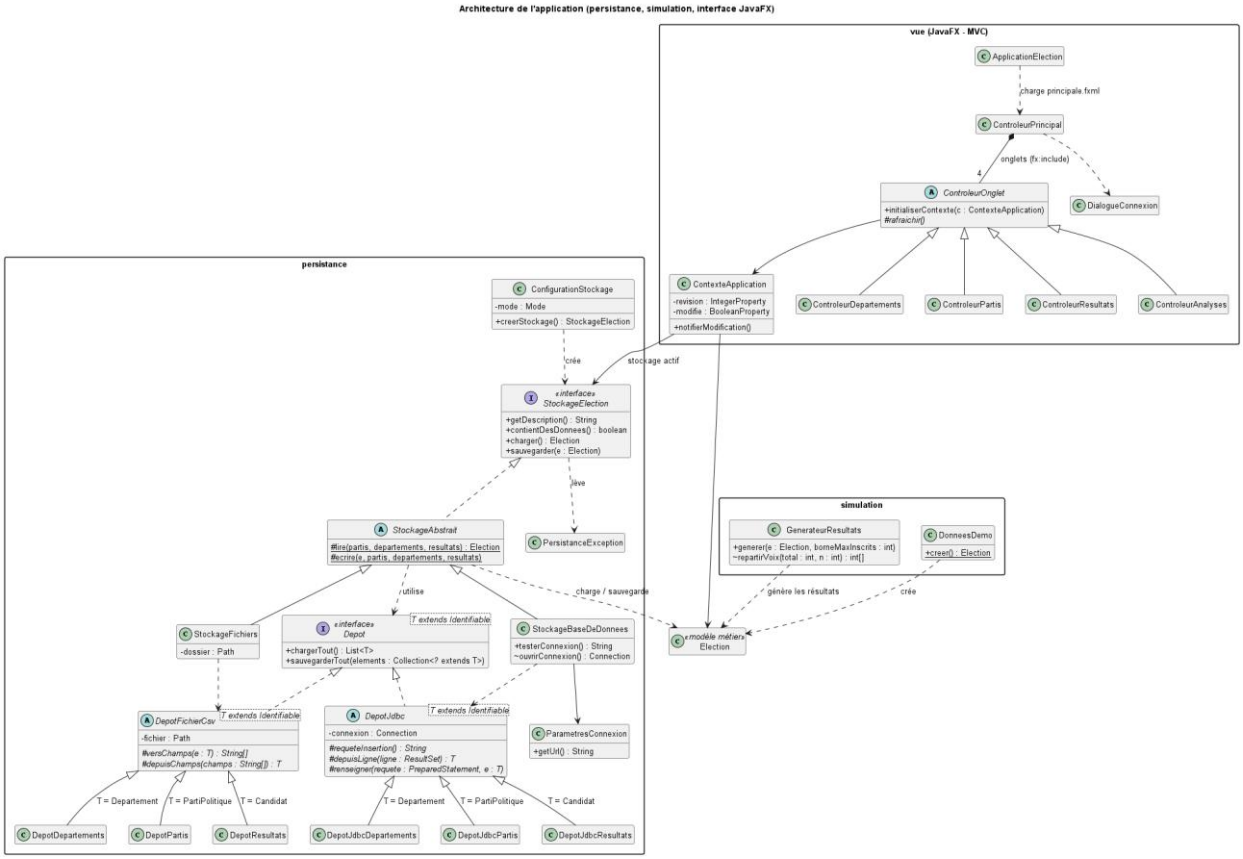


Figure 2 — Diagramme de classes de l'architecture (persistance, simulation, interface JavaFX)

La partie persistance montre les deux familles de dépôts génériques (`DepotFichierCsv<T>` et `DepotJdbc<T>`), chacune spécialisée pour les trois types `Departement`, `PartiPolitique` et `Candidat`. Les fichiers PNG en pleine résolution sont fournis dans le dossier `docs/` du projet.

4. Réalisation

4.1 Environnement de développement

Élément	Choix
Langage	Java 17 ou plus récent (développé et testé avec Temurin 21)
Interface graphique	JavaFX 21 (FXML, CSS, graphiques)
Construction	Maven, avec le wrapper `mvnw` (aucune installation de Maven nécessaire)
Base de données (facultative)	MySQL / MariaDB via XAMPP, pilote `mysql-connector-j`
Tests	JUnit 5
IDE	Visual Studio Code (extensions Java)

Le fichier `election-nepalus.jar` produit par `mvnw clean package` contient JavaFX (pour Windows, Linux et macOS) et le pilote MySQL : il se lance avec un simple `java -jar`.

4.2 Modèle métier et respect des règles

Toute saisie de résultats passe par `Election.enregistrerResultats(...)`, qui applique les valeurs puis les fait valider par la règle de scrutin. Si la règle est violée, **les anciennes valeurs sont restaurées** et une `ElectionException` explique l'erreur à l'utilisateur : le modèle n'est jamais laissé dans un état incohérent. Voici la validation du scrutin majoritaire à un tour :

```
public void valider(Departement d) throws ElectionException {
    if (d.getVotants() > d.getInscrits()) {
        throw new ElectionException("... plus de votants que d'inscrits");
    }
    if (d.getTotalVoix() != d.getVotants()) {      // pas de vote blanc
        throw new ElectionException("... le total des voix doit être égal au nombre de votants");
    }
    Set<Integer> voixDejaVues = new HashSet<>();
    for (Candidat c : d.getCandidats()) {          // pas d'égalité de voix
        if (!voixDejaVues.add(c.getVoix())) {
            throw new ElectionException("... deux candidats ne peuvent pas obtenir le même nombre
de voix");
        }
    }
}
```

Les questions d'analyse sont écrites avec des **streams** et des **expressions lambda**. Par exemple :

```
public List<Departement> departementsUnanimes() {
    return departements.values().stream()
            .filter(Departement::isDepouille)
            .filter(d -> d.getVotants() == d.getInscrits())
            .filter(d -> d.getCandidats().stream().anyMatch(c -> c.getVoix() == d.getVotants()))
            .collect(Collectors.toList());
}
```

Les départements et les partis sont rangés dans des `TreeMap` indexées par leur code : les listes sont ainsi toujours triées (par numéro de département, par sigle) et la recherche d'un élément est directe.

4.3 Génération aléatoire des résultats

Pour chaque département, `GenerateurResultats` tire le nombre d'inscrits entre le minimum $n(n-1)/2$ et la borne choisie, puis le nombre de votants entre ce minimum et le nombre d'inscrits.

La répartition des voix utilise la **méthode des points de coupure** : on tire $n-1$ nombres au hasard dans l'intervalle [0, votants], on les trie, et les écarts successifs entre 0, ces points et le total donnent les

voix des *n* candidats. La somme est donc exactement égale au nombre de votants (pas de vote blanc). Si le tirage contient une égalité, on recommence, dans la limite de 1 000 essais. Au-delà, on part de la répartition 0, 1, …, *n* − 1, on donne le reste au dernier candidat et on mélange, ce qui garantit des scores distincts.

Le tirage est représenté par la classe interne statique `Tirage`, qui **ne modifie pas le modèle**. Cela permet de calculer tous les tirages dans un thread d'arrière-plan, puis de les appliquer à l'élection dans le thread JavaFX (voir 4.6).

4.4 Persistance des données

L'utilisateur choisit le support dans le menu *Stockage*. Les deux supports implémentent la même interface `StockageElection` : le reste de l'application ne sait pas lequel est actif.

Fichiers CSV (demandés par l'énoncé)

Les données sont enregistrées dans trois fichiers du dossier `donnees/` : `partis.csv`, `departements.csv` et `resultats.csv` (une ligne par candidat : numéro de département, sigle du parti, voix). La lecture et l'écriture utilisent `java.nio.file.Files` en UTF-8.

Base de données MySQL (complément)

La base `projets_java` (sur `localhost:3306`, utilisateur `root` par défaut sous XAMPP) contient trois tables, **créées automatiquement** si elles n'existent pas :

Table	Colonnes et contraintes
`parti`	`sigle` (clé primaire), `nom`
`departement`	`numero` (clé primaire), `nom`, `inscrits`, `votants` ; contrôle `votants <= inscrits`
`resultat`	`numero_departement`, `sigle_parti` (clé primaire composée, clés étrangères avec suppression en cascade), `voix`

Les accès utilisent des `PreparedStatement` (requêtes paramétrées, exécutées par lots). L'enregistrement complet se fait dans une **transaction** : en cas d'erreur, un `rollback` laisse la base inchangée.

```
connexion.setAutoCommit(false);
try {
    ecrire(election, new DepotJdbcPartis(connexion),
                     new DepotJdbcDepartements(connexion),
                     new DepotJdbcResultats(connexion, election));
    connexion.commit();
} catch (PersistanceException | RuntimeException e) {
    connexion.rollback();
    throw e;
}
```

Lors d'un changement de stockage, l'application propose soit de **charger** les données de la nouvelle source, soit d'y **copier** l'élection affichée (migration CSV ⇄ MySQL). Au premier lancement, elle essaie la base MySQL et, si aucun serveur ne répond, utilise les fichiers CSV sans afficher d'erreur. Le choix est mémorisé dans `election.properties`.

4.5 Interface graphique

Au démarrage, un écran d'accueil s'affiche pendant le chargement des données. La fenêtre principale comporte une barre de menus (*Fichier*, *Stockage*, *Simulation*, *Aide*), un bandeau d'indicateurs (nombre de départements, de partis, taux de participation général, parti en tête), quatre onglets et une barre d'état qui indique le stockage actif.

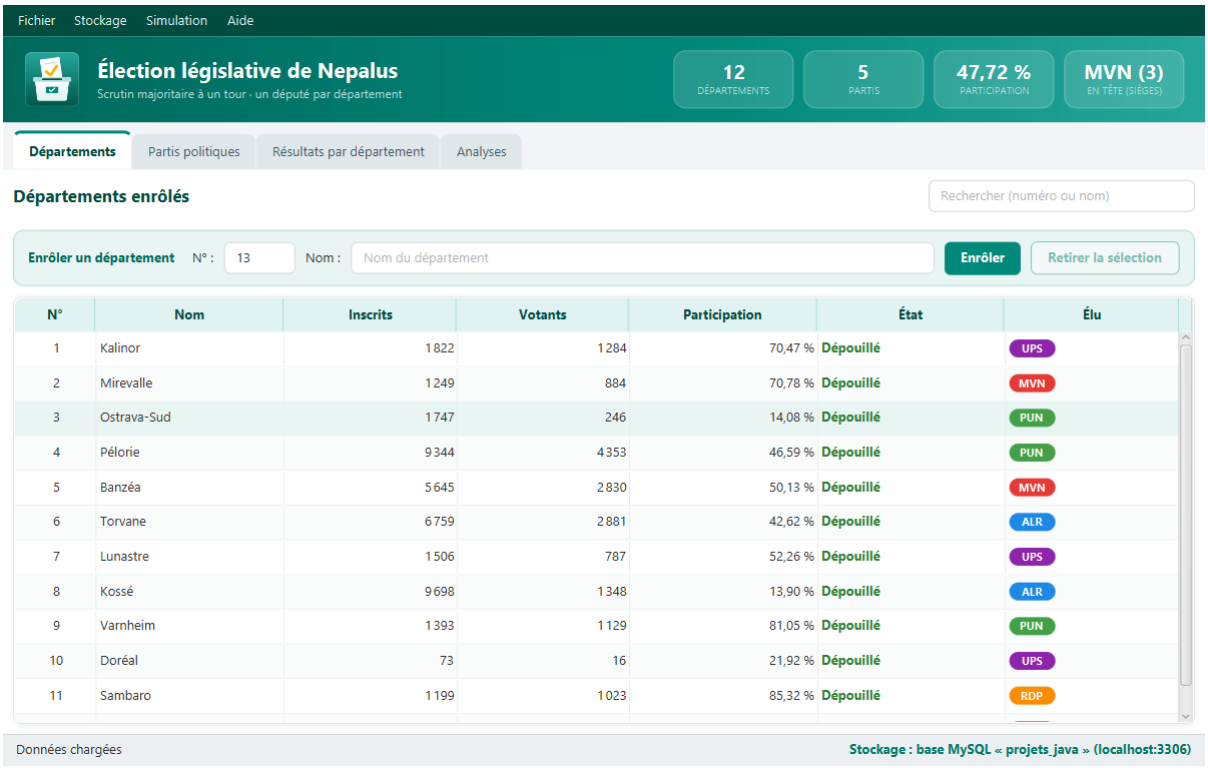


Figure 3 — Onglet « Départements » : enrôlement, inscrits, votants, participation, état et élu

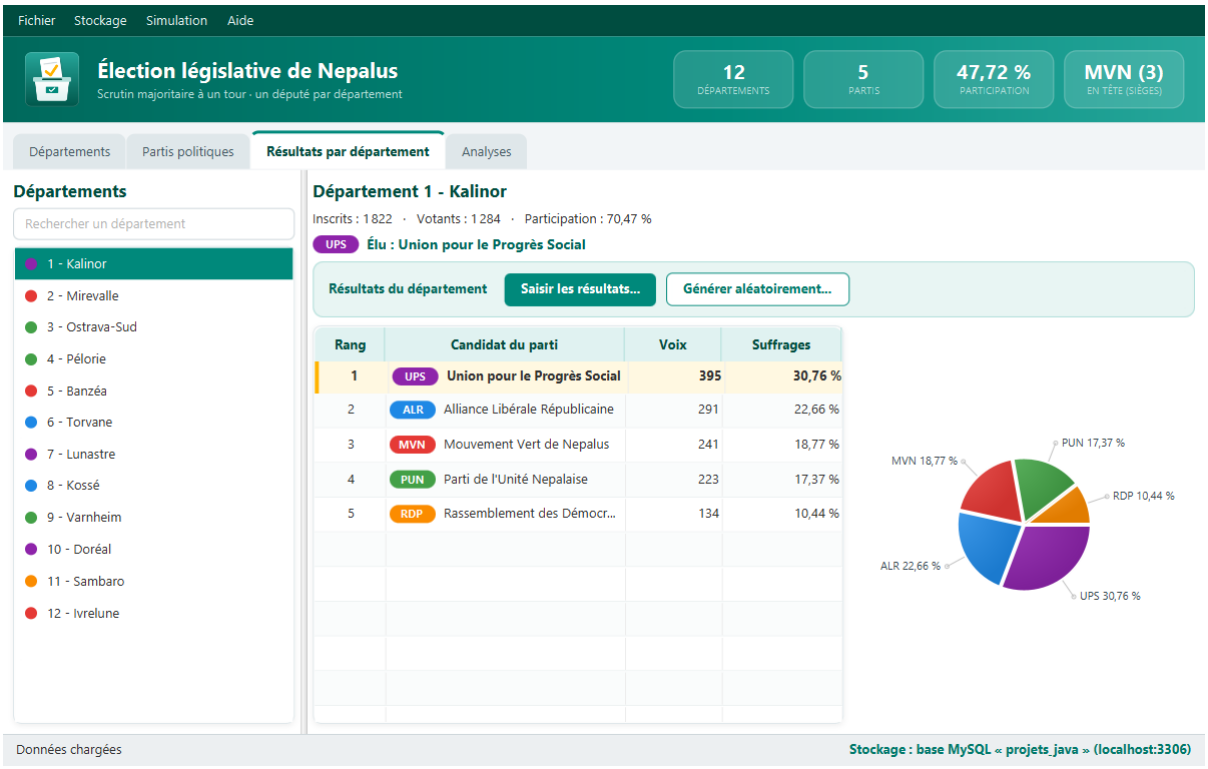


Figure 4 — Onglet « Résultats par département » : classement des candidats, élu et répartition des suffrages

Dans l'onglet *Résultats par département*, le bouton *Saisir les résultats…* ouvre un formulaire de saisie manuelle (inscrits, votants, voix de chaque candidat), validé par la règle de scrutin. Le bouton *Générer aléatoirement…* tire les résultats du seul département sélectionné.

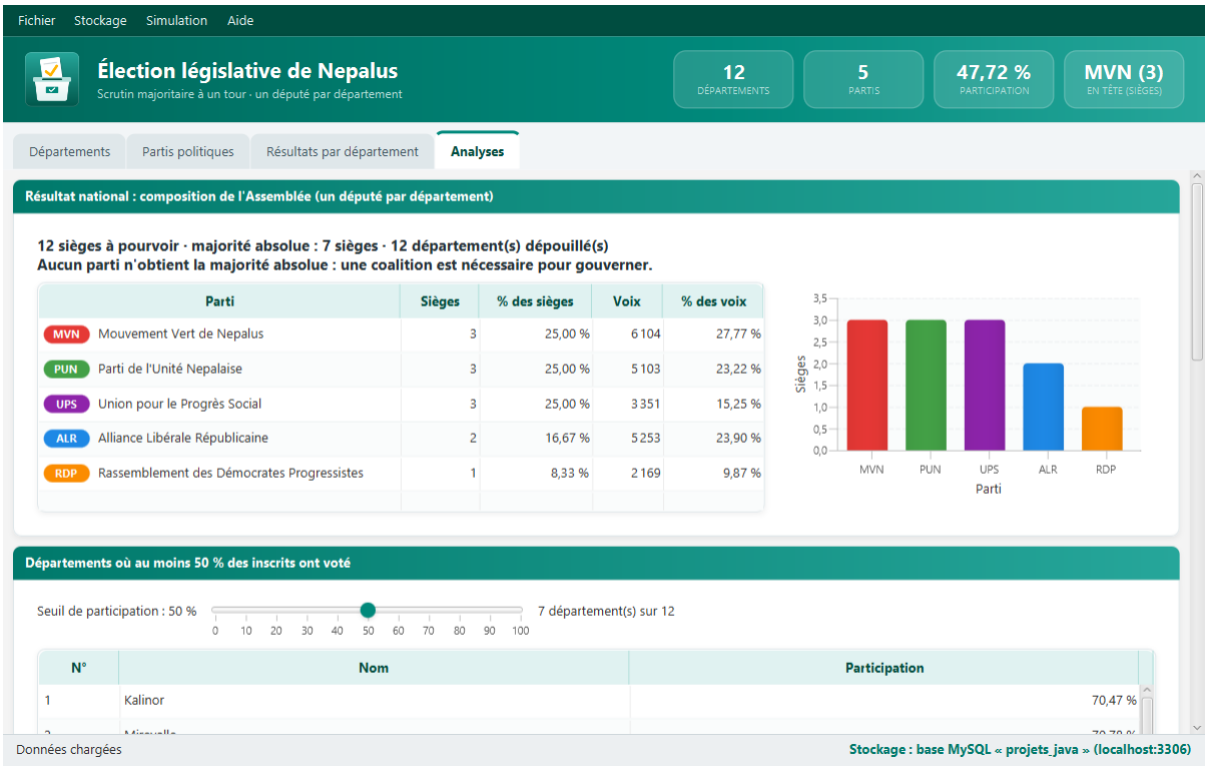


Figure 5 — Onglet « Analyses » : résultat national et départements où au moins 50 % des inscrits ont voté

L'onglet *Analyses* regroupe les réponses aux questions de l'énoncé dans des sections repliables : résultat national (sièges, majorité absolue, voix), participation, parti gagnant et parti le plus mal classé par département, départements au-dessus d'un seuil de participation, départements unanimes, et statistiques de participation avec histogramme.

Les graphiques sont **interactifs** : un clic sur la barre d'un parti filtre la liste sur les départements qu'il a gagnés, un clic sur une barre de l'histogramme liste les départements de cette tranche de participation, et un double-clic sur un département ouvre ses résultats.

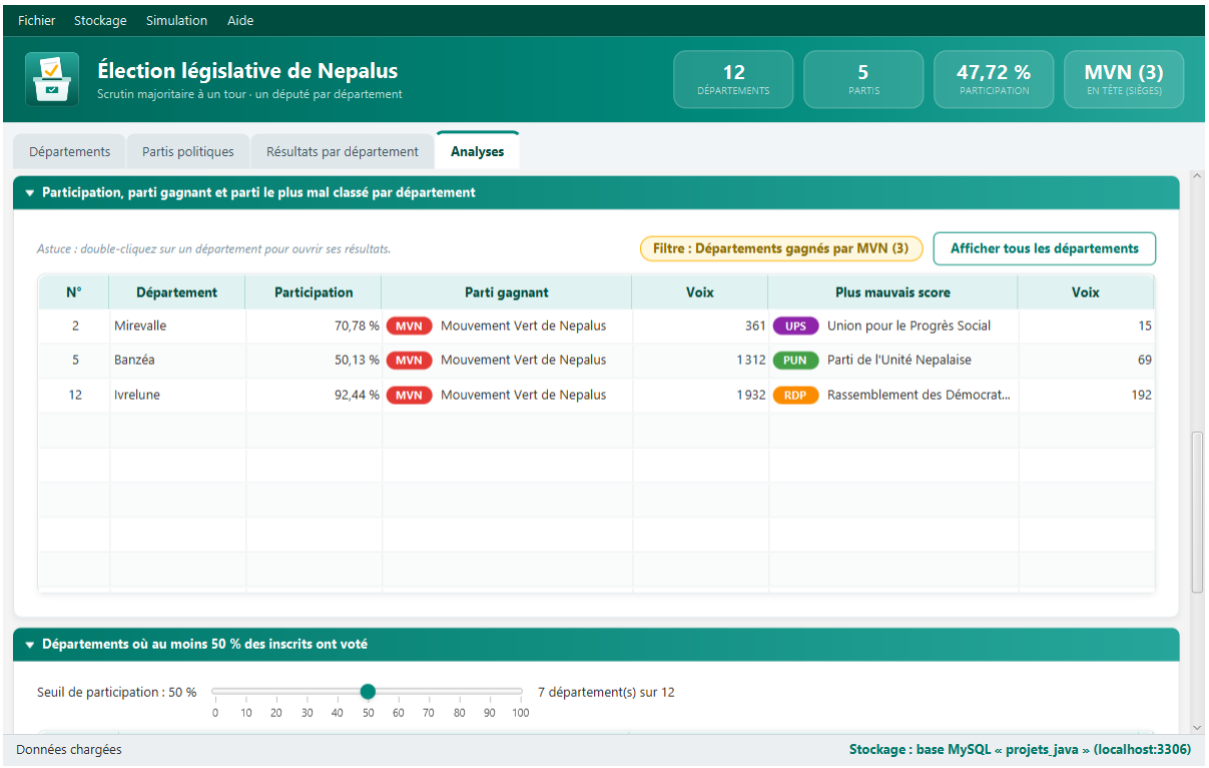


Figure 6 — Après un clic sur la barre du parti MVN : départements gagnés par ce parti, avec le parti gagnant et le parti le plus mal classé

4.6 Traitements en arrière-plan

La génération des résultats et les accès au stockage (en particulier à la base de données) peuvent prendre du temps. Ils sont exécutés dans une `Task` JavaFX lancée dans un `Thread` séparé, avec une barre de progression, afin que la fenêtre reste réactive. Le modèle n'est modifié que dans le thread JavaFX, une fois la tâche terminée :

```
Task<List<Tirage>> tache = new Task<>() {          // classe anonyme
    @Override
    protected List<Tirage> call() throws ElectionException {
        List<Tirage> tirages = new ArrayList<>();
        for (int i = 0; i < departements.size(); i++) {
            tirages.add(generateur.tirer(departements.get(i), borne));
            updateProgress(i + 1, departements.size());
        }
        return tirages;
    }
};
```

Le rafraîchissement des onglets repose sur les **propriétés observables** de `ContexteApplication` : chaque modification du modèle incrémente une propriété « révision » que tous les contrôleurs écoutent. De même, le titre de la fenêtre et la barre d'état sont liés (*binding*) aux propriétés « données modifiées » et « stockage actif ».

5. Notions du cours mises en œuvre

Notion	Mise en œuvre dans le projet
Interfaces, classes abstraites, héritage, polymorphisme	`Identifiable`, `Entite`, `RegleScrutin`, `StockageElection`, `Depot`, `ControleurOnglet` (voir section 3)
Classes internes et classes anonymes	Classe interne `GenerateurResultats.Tirage` ; énumération interne `ConfigurationStockage.Mode` ; `Task` anonymes ; cellules de `TableView` et `ListView` (`updateItem` redéfinie) ; implémentation anonyme de `Navigation`
Expressions lambda, références de méthode, streams	Questions d'analyse de `Election`, comparateurs (`Comparator.comparingInt(Candidat::getVoix)`), gestionnaires d'événements des contrôleurs
Généricité (bornée, jokers, méthodes génériques)	`Depot<T extends Identifiable>`, `Collection<? extends T>`, méthodes génériques `<S, T>` de `ControleurOnglet` pour les colonnes et le filtrage
Collections	`List`, `Map`, `TreeMap`, `LinkedHashMap`, `Set`/`HashSet`, `Optional` ; collections JavaFX `ObservableList`, `FilteredList`, `SortedList`
Exceptions	Exceptions contrôlées `ElectionException` et `PersistanceException` ; *try-with-resources* sur les connexions et fichiers
Entrées / sorties (fichiers)	`java.nio.file` : lecture et écriture des fichiers CSV et du fichier `election.properties`
JDBC	`DriverManager`, `Connection`, `PreparedStatement` (paramètres et lots), `ResultSet`, transactions (`commit`/`rollback`), `DatabaseMetaData`
Threads	`Task` exécutée dans un `new Thread(...)` pour la génération et les accès au stockage
Annotations	`@FXML`, `@Override`, annotations JUnit (`@Test`, `@BeforeEach`, …)
JavaFX	FXML et MVC avec `fx:include`, `TableView`, `ListView`, graphiques (`BarChart`, `PieChart`), feuille de style CSS, propriétés observables et *binding*, dialogues, animations

6. Tests et validation

Le projet comporte **27 tests unitaires JUnit 5**, lancés par `mvnw test` :

Classe de test	Tests	Ce qui est vérifié
`ElectionTest`	11	Un candidat par parti dans chaque département ; refus des doublons ; enregistrement de résultats valides ; refus d'un vote blanc, d'une égalité de voix et de votants plus nombreux que les inscrits ; seuil de 50 % de participation ; unanimité impossible avec trois partis et détectée avec deux ; sièges par parti ; effacement des résultats quand un nouveau parti est enrôlé
`ResultatNationalTest`	4	Majorité absolue ; absence de majorité quand les sièges sont partagés ; statistiques de participation, avec et sans données
`GenerateurResultatsTest`	3	Sur 200 tirages : total des voix = votants, voix toutes distinctes, votants ≤ inscrits ≤ borne ; répartition distincte même avec très peu de voix ; refus d'une borne trop faible
`StockageElectionTest`	4	Enregistrement puis rechargement à l'identique dans les fichiers CSV ; dossier vide ; fichier corrompu signalé ; configuration du stockage enregistrée et relue
`StockageBaseDeDonneesTest`	5	Base neuve vide ; enregistrement puis rechargement MySQL ; un second enregistrement remplace le premier ; test de connexion ; mauvais mot de passe signalé. Ces tests utilisent une base séparée `projets_java_test`, pour ne jamais toucher aux données réelles, et sont ignorés si aucun serveur MySQL n'est démarré

Lors de la dernière exécution, MySQL étant arrêté, les 22 tests sans base de données ont réussi et les 5 tests MySQL ont été ignorés. Avec XAMPP démarré, les 27 tests s'exécutent.

En plus de ces tests automatiques, l'application a été testée manuellement : données de démonstration, génération aléatoire, saisie manuelle de résultats invalides (refusés avec un message clair), enregistrement et rechargement dans les deux stockages, migration CSV ⇄ MySQL, puis lancement du fichier `.jar` final.

7. Guide d'utilisation

7.1 Lancement

Il suffit d'un Java 17 ou plus récent. Dans une invite de commandes, depuis le dossier qui contient le fichier :

```
java -jar election-nepalus.jar
```

Pour reconstruire l'archive et lancer les tests depuis les sources (dossier `election-nepalus`, sous Windows) :

```
.\mvnw.cmd test             # lancer les tests
.\mvnw.cmd clean package    # produire target\election-nepalus.jar
.\mvnw.cmd javafx:run       # lancer l'application depuis les sources
```

Le stockage MySQL est facultatif. Pour l'utiliser, démarrer MySQL dans XAMPP et créer la base `projets_java` dans phpMyAdmin : les tables sont créées par l'application.

7.2 Scénario d'utilisation

1. **Fichier > Données de démonstration** charge 12 départements et 5 partis. On peut aussi les enrôler un par un dans les onglets *Départements* et *Partis politiques*.
2. **Simulation > Générer les résultats aléatoirement…** demande la borne supérieure du nombre d'inscrits par département, puis tire tous les résultats.
3. L'onglet **Résultats par département** affiche le détail d'un département. On peut y saisir des résultats à la main.
4. L'onglet **Analyses** répond aux questions de l'énoncé.
5. **Fichier > Enregistrer** (Ctrl+S) enregistre l'élection dans le stockage actif, indiqué en bas à droite de la fenêtre. Le menu **Stockage** permet de passer des fichiers CSV à la base MySQL, et inversement.

8. Difficultés rencontrées, limites et perspectives

8.1 Difficultés rencontrées

- **Générer des voix sans égalité et sans vote blanc.** Une répartition naïve produit souvent des égalités, surtout dans les petits départements. La méthode des points de coupure, avec une solution de secours garantie, et le calcul du nombre minimal de votants ont résolu ce problème (section 4.3).
- **La question des départements « unanimes ».** Nous avons constaté qu'elle est incompatible avec la règle « pas d'égalité » dès trois partis (section 2.5). Nous avons choisi de calculer la réponse fidèlement plutôt que de contourner la règle, et de le vérifier par un test avec deux partis.
- **Garder l'interface réactive.** Les calculs et les accès à la base dans le thread JavaFX figeaient la fenêtre. Nous les avons déplacés dans des `Task`, en séparant le tirage (en arrière-plan) de la modification du modèle (dans le thread JavaFX).
- **Cohérence des données enregistrées.** Une erreur au milieu d'un enregistrement en base pouvait laisser des données partielles. L'utilisation d'une transaction a réglé ce point.
- **Construction de l'archive.** L'éditeur compile lui aussi le projet dans `target/classes`, et une classe compilée au milieu d'une modification s'est un jour retrouvée dans le `.jar`. Depuis, l'archive est toujours produite par `mvnw clean package`, puis testée par un vrai lancement.

8.2 Limites

- Une seule élection est gérée à la fois. Le stockage ne conserve pas d'historique des scrutins.
- Le mot de passe MySQL est enregistré en clair dans `election.properties`. Ce fichier ne doit donc pas être diffusé.
- Le tirage aléatoire suit une loi uniforme et ne reproduit pas de tendances électorales réalistes.

8.3 Perspectives

- Ajouter d'autres règles électorales (scrutin à deux tours, proportionnelle) en implémentant simplement l'interface `RegleScrutin`.
- Conserver plusieurs élections et comparer leurs résultats dans le temps.
- Exporter les analyses (PDF, tableur) et proposer une carte des départements.

9. Conclusion

Ce projet nous a permis de réaliser une application complète qui simule le traitement d'une élection législative dans le pays de Nepalus. L'application enrôle les départements et les partis, génère ou enregistre les résultats en respectant les règles imposées (pas de vote blanc, pas d'égalité, scrutin majoritaire à un tour) et répond, dans une interface JavaFX, à toutes les questions de l'énoncé, ainsi qu'à plusieurs questions supplémentaires (composition de l'Assemblée, majorité absolue, statistiques de participation).

Sur le plan de la conception, nous avons cherché à mettre en pratique les notions du cours : interfaces et classes abstraites pour séparer les responsabilités, généricité pour écrire une seule fois l'accès aux données, collections et streams pour les analyses, classes internes et anonymes, threads, et JavaFX avec FXML, MVC et propriétés observables. La persistance par fichiers demandée par l'énoncé a été complétée par une persistance en base MySQL, qui réutilise les notions JDBC des travaux dirigés.

Enfin, l'analyse des règles nous a rappelé l'importance de l'étude du problème avant le développement : la contradiction entre l'unanimité et l'absence d'égalité, ou le nombre minimal de votants, n'apparaissent qu'en raisonnant sur les règles elles-mêmes.